

Cleaning-World

Languages and Compilers – Final Project Report

1. Introduction

The objective of this project is to design and implement a complete language processing pipeline for a small domain-specific language called **Cleaning-World**. The language allows users to define a world and specify the behavior of a cleaning agent using variables, control structures, functions, and built-in robot actions.

The project was developed incrementally through five parts, covering:

- Language design and lexical analysis
- Parsing and grammar implementation
- Static semantic analysis
- CST to AST transformation
- Execution through direct interpretation

This report explains how each phase works, the design choices made, and how feedback was incorporated to improve the final system.

2. Design Philosophy

From the beginning, we aimed to implement the **simplest possible language that fully works**, rather than maximizing feature complexity.

Our guiding principles were:

- correctness over feature richness,
- clarity over cleverness,
- completeness over theoretical ambition.

This allowed us to successfully implement and integrate **all required compiler phases (Parts 1–5)** into a coherent and executable system.

3. Language Design (Part 1)

3.1 Program Structure

Each program consists of:

- a single WORLD definition,
- optional FUNC definitions,
- a single AGENT definition.

We deliberately chose a **single-file program model**. Supporting multiple files or libraries was unnecessary for the Cleaning World task and would have added complexity to parsing and execution without pedagogical benefit.

3.2 World Initialization

The world is defined directly within the program rather than being loaded from an external file.

Rationale:

- keeps programs self-contained,
- avoids introducing file I/O semantics,
- simplifies parsing, semantic analysis, and interpretation,
- makes testing easier and more transparent.

3.3 Control Structures

The language supports:

- selection via IF / ELSE / ENDIF,
- repetition via WHILE / DO / ENDWHILE.

Iteration was chosen instead of recursion for looping constructs because:

- it is sufficient for all required agent behaviors,
- it maps cleanly to both the AST and interpreter,
- it avoids unnecessary execution complexity.

3.4 Functions

Functions are designed with:

- parameters passed **by value**,
- return values restricted to **simple types (integers)**.

Our implementation supports function calls using a stack-based execution model.

Our example programs focus on **non-recursive functions**, since recursion is not required for the Cleaning World task.

We intentionally avoided more advanced features such as structured return types or complex parameter passing mechanisms in order to keep semantic analysis and execution manageable.

3.5 TURN, LEFT, RIGHT Clarification (Addressing Feedback)

Based on instructor feedback, we clarified the syntax and semantics of turning actions:

- LEFT and RIGHT are **arguments to the TURN command**, not standalone commands.

This mirrors how DIRT and OBSTACLE are used as arguments to SENSE, improving conceptual consistency across the language.

4. Lexical Analysis (Part 2)

4.1 Tool Choice

We implemented the lexer using **PLY (Python Lex)**.

Rationale:

- allows concise and readable regular-expression-based token definitions,
- integrates naturally with the parser,
- provides line-number tracking for error reporting,
- is well suited for incremental development and debugging.

4.2 Token Organization and Maintainability (Addressing Feedback)

Token definitions, reserved words, and token identifiers are centralized in a dedicated module (`tokens.py`), which is imported by the lexer.

This design avoids unnecessary duplication and reduces maintenance risks if the language evolves. While early iterations involved some repetition, later refinements ensured consistency and clarity in token management.

4.3 Symbol Table Initialization

The lexer initializes the symbol table with reserved words and adds user-defined identifiers as they are encountered. This early symbol tracking supports later semantic checks while keeping the lexer's responsibilities limited.

4.4 Lexer Output and Testing

To demonstrate completeness and correctness:

- we created multiple test programs exercising all lexemes and operators,
- we documented coverage using a **line-based testing table** that shows exactly where each token and grammar construct is tested.

This explicitly addresses feedback regarding lexer test completeness.

5. Parsing and Concrete Syntax Tree (Part 3)

5.1 Parser Choice

We used the **LALR(1) parser provided by PLY (yacc)**.

Rationale:

- supports grammars that are not strictly LL(1),
- avoids rewriting productions to enforce pairwise-disjoint RHSs,
- directly addresses feedback regarding non-disjoint productions such as `<stmt_list>`, `<condition>`, and `<arg_list>`.

This choice allowed us to preserve a clear and expressive grammar without sacrificing correctness.

5.2 Grammar Coverage

We ensured that:

- every production,
- and every alternative RHS

is exercised by at least one example program.

Coverage is documented using a table that maps grammar constructs to specific files and line numbers.

5.3 CST Generation

When parsing succeeds, the parser generates a **Concrete Syntax Tree (CST)**.

Readable CSTs are written to files for each test program, using consistent naming.

6. Static Semantics and AST Construction (Part 4)

6.1 Static Semantic Checks

The static semantic analyzer performs checks such as:

- declaration-before-use,
- duplicate declarations,
- correct number of function arguments,
- basic scope correctness.

Errors are collected and reported before execution, preventing invalid programs from running.

6.2 CST to AST Conversion

The CST is transformed into an **Abstract Syntax Tree (AST)** by:

- removing punctuation and purely syntactic nodes,
- keeping only semantically meaningful constructs.

This simplification makes execution clearer and reduces interpreter complexity.

Readable ASTs are generated for each test program and kept consistent with their CSTs.

7. Direct Interpretation and Execution (Part 5)

7.1 Execution Pipeline

The complete pipeline is implemented in `run_complete.py`, which:

1. parses the source file into a CST,
2. performs semantic analysis and builds the AST,
3. executes the AST using the interpreter.

This script provides a single entry point that integrates all project parts.

7.2 Interpreter Design

The interpreter traverses the AST and evaluates:

- expressions and assignments,
- control structures,
- function calls,
- built-in robot commands.

Function calls are handled using a **stack-based execution model**, where each call creates a new activation record with its own local variables. Our example programs avoid complex cross-scope interactions in order to keep execution behavior simple and predictable.

8. Testing and Examples

Three main example programs are provided:

- test_conditionals.cl
- test_loops.cl
- test_functions.cl

These programs collectively test:

- relational and boolean expressions,
- control flow,
- function definition and invocation.

Each program successfully runs through all phases of the pipeline, from lexical analysis to execution.

9. Incorporation of Feedback and Improvements

Based on instructor feedback, we:

- clarified ambiguous language constructs,
- expanded example coverage,
- documented lexer and parser testing more explicitly,
- reflected on maintainability and design trade-offs.

These changes improved clarity, robustness, and alignment with project requirements.

10. Conclusion

This project demonstrates a complete and functional implementation of a small programming language, following classical compiler design principles.

By prioritizing simplicity, correctness, and integration, we were able to:

- implement all required phases,
- connect them into a working system,
- and respond constructively to feedback.